

Add settings to your extension

Previous: [Adding Dock support](#)

Extensions can provide their own settings page, letting users configure extension behavior directly within the Command Palette. The settings helpers in the toolkit make it easy to define toggles, text fields, and choice sets without writing UI code.

How extension settings work

The toolkit provides a [Settings](#) class that manages a collection of typed settings. You define the settings you need, convert them to content for display, and react to changes through an event. Command Palette renders the settings form automatically.

Add a settings page

1. In the `Pages` directory, add a new class
2. Name the class `MySettingsPage.cs`
3. Update the file to:

C#

```
using Microsoft.CommandPalette.Extensions;
using Microsoft.CommandPalette.Extensions.Toolkit;

internal sealed partial class MySettingsPage : ContentPage
{
    private readonly Settings _settings = new();

    public MySettingsPage()
    {
        Name = "Settings";
        Icon = new IconInfo("\uE713"); // Settings gear icon
        Title = "Extension Settings";

        _settings.Add(new ToggleSetting("notifications", true)
        {
            Label = "Enable notifications",
            Description = "Show a notification when a task completes",
        });
        _settings.Add(new TextSetting("defaultQuery", string.Empty)
```

```

    {
        Label = "Default search query",
        Description = "Pre-fill the search box with this text",
    });

    _settings.SettingsChanged += OnSettingsChanged;
}

public override IContent[] GetContent()
{
    return _settings.ToContent();
}

private void OnSettingsChanged(object sender, Settings args)
{
    // Read updated values
    var notificationsEnabled = _settings.GetSetting<bool>("notifications");
    var defaultQuery = _settings.GetSetting<string>("defaultQuery");
}
}

```

Available setting types

[Expand table](#)

Type	Description	Default value type
ToggleSetting	A checkbox / toggle switch	bool
TextSetting	A single-line text input	string
ChoiceSetSetting	A drop-down list of choices	string (selected value)

Choice set example

```

C#

var choices = new List<ChoiceSetSetting.Choice>
{
    new("Small", "sm"),
    new("Medium", "md"),
    new("Large", "lg"),
};

_settings.Add(new ChoiceSetSetting("size", choices)
{
    Label = "Result size",

```

```
Description = "Choose how results are displayed",  
});
```

Read setting values

Use `GetSetting<T>` to read a setting value by key at any time:

C#

```
var isEnabled = _settings.GetSetting<bool>("notifications");
```

Use `TryGetSetting<T>` when the setting might not exist:

C#

```
if (_settings.TryGetSetting<string>("defaultQuery", out var query))  
{  
    // use query  
}
```

React to changes

Subscribe to the `SettingsChanged` event to be notified when the user modifies any setting:

C#

```
_settings.SettingsChanged += (sender, args) =>  
{  
    // Refresh your extension state based on the new settings  
};
```

Wire the settings page to your extension

To make the settings page accessible from the Command Palette extension manager, expose it from your `ListPage` as a list item, or wire it through your command provider.

C#

```
new ListItem(new MySettingsPage())  
{  
    Title = "Settings",
```

```
    Subtitle = "Configure extension options",  
    Icon = new IconInfo("\uE713"),  
}
```

Related content

- [Settings class reference](#)
- [ICommandSettings interface](#)
- [Extension samples](#)

Last updated on 04/16/2026